Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY, BENGALURU

(An Autonomous Institute, Affiliated to VTU, Approved by AICTE, Accredited by NAAC with 'A+' Grade)
Near Jnana Bharathi Campus, Bengaluru – 560056

MASTER OF COMPUTER APPLICATIONS PROGRAMME

(Accredited by National Board of Accreditation)

A MINI PROJECT REPORT

ON

"Brick Breaker Arena Game – An Interactive Java Mini Project"

Submitted in partial fulfillment of the requirement of 3rd Semester for the award of the Degree of

Master of Computer Applications

By

SHARANYA N

1DA24MC045

Under the Guidance of

DR. DHARANI N V

Assistant Professor, Dept. of MCA, Dr. AIT, Bengaluru.

Oct 2025 – Feb 2026

Visvesvaraya Technological University

Jnana Sangama, Belagavi, Karnataka – 590018

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY, BENGALURU

(An Autonomous Institute, Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi, Accredited by NAAC with 'A+' Grade)

Bengaluru - 560056

Aided By Govt. of Karnataka

MASTER OF COMPUTER APPLICATIONS PROGRAMME

(Accredited by National Board of Accreditation)

CERTIFICATE

This is to certify that **SHARANYA N** bearing **1DA23MC045** has completed Mini Project-2 (MCAM25) entitled "Brick Breaker Arena Game – An Interactive Java Mini Project" as a partial fulfilment of 3rd Semester for the award of Master of Computer Applications degree, during the academic year 2025-26 under our supervision.

Signature of the Guide	Signature of the HOP	Signature of the Principal
Dr. Dharani N V	Dr. L. Manjunatha Rao	Dr. Mahalinga V Mandi
Assistant Professor	Professor and Head	Principal
MCA Programme	MCA Programme	Dr. AIT

Viva – Voice Examination

Name and Signature of the Examiners

1.

2.

Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY, BENGALURU

(An Autonomous Institute, Affiliated to VTU, Belagavi, Approved by AICTE, New Delhi, Accredited by NAAC with 'A+' Grade)

Bengaluru - 560056

Aided By Govt. of Karnataka

DECLARATION

I **SHARANYA N** student of 2nd year MCA, **Dr. AMBEDKAR INSTITUTE OF TECHNOLOGY,** bearing USN **1DA24MC045** hereby declare that the Mini Project entitled "**BRICK BREAKER ARENA GAME—AN INERACTIVE JAVA MINI PROJECT**" has been carried out by me under the supervision of Guide **Dr. Dharani N V** Assistant Professor and **Dr. L. Manjunatha Rao**, Professor & Head MCA Programme submitted in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications during the academic year 2025-26.

Signature of the Student
SHARANYA N
(1DA24MC045)

ACKNOWLEDGEMENT

I would like to thank **Dr. Mahalinga V Mandi,** Principal, Dr. AIT, who has always been a great source of inspiration and for permitting me to carry out the Mini Project.

I thank **Dr. L. Manjunatha Rao,** Professor and Head, MCA Programme, for his kind cooperation. His guidance, motivation, and insightful suggestions have greatly enriched my learning experience and helped shape the direction of this work.

I extend my special thanks to **Mrs. Dharani N V** Assistant Professor, MCA Programme, she has been constant source of inspiration in completing the Mini Project with her constant guidance, patience, and mentorship.

I also extend my sincere thanks to all the faculty members of the Department of MCA for their cooperation and support throughout my academic journey.

Place: Bengaluru

Sharanya

(1DA24MC045)

CONTENTS

1. **INTRODUCTION** 1-2

2. **SYSTEM OVERVIEW** 3-4

3. **HARDWARE REQUIREMENTS** 5-7

4. **SOFTWARE REQUIREMENTS** 8-9

5. **SYSTEM ARCHITECTURE** 10-12

6. **PROJECT ALGORITHM** 13-17

7. **IMPLEMENTATION** 18-26

8. **CONCLUSION** 27

CHAPTER 1: INTRODUCTION

1.1 General Introduction

In recent years, computer-based games have become an important part of entertainment and learning. Simple 2D games help in understanding fundamental concepts of programming, graphics, and user interaction. Game development provides practical exposure to logical thinking, real-time processing, and event-driven programming.

Traditional learning methods often focus only on theory, which makes it difficult for students to understand how concepts work in real-time applications. To overcome this, developing interactive games using programming languages like Java helps students gain hands-on experience and better understanding of core programming concepts.

This project, **Breakout Ball Game**, is developed using **Java Swing and AWT** to demonstrate graphical user interface design and game logic implementation. The game involves controlling a paddle to bounce a ball and break bricks arranged on the screen. The project focuses on concepts such as **object-oriented programming, event handling, collision detection, timers, and graphics rendering**.

By developing this project, learners can understand how Java can be used to build **interactive and real-time applications**, making it an effective educational tool as well as an entertaining game.

1.2 Motivation

The motivation behind this project is to gain practical knowledge of **Java programming through game development**. While theoretical learning provides an understanding of concepts, implementing a real-time game helps in applying those concepts in a meaningful and engaging way.

Many beginners find it difficult to understand topics such as **event handling, graphics rendering, collision detection, and timer-based execution**. Developing a simple arcade-style game like the Breakout Ball Game provides a clear and interactive way to learn these concepts.

This project is motivated by the need to create a **simple, low-complexity, and interactive Java application** that demonstrates the effective use of **Java Swing and AWT**. By building this game, students can understand how Java can be used to develop real-time applications, improve logical thinking, and enhance problem-solving skills, making learning both effective and enjoyable.

1.3 Problem Statement

Most beginner-level Java applications focus only on console-based programs, which do not provide real-time interaction or graphical representation. As a result, students often find it difficult to understand how core Java concepts such as **event handling, graphics, timers, and collision logic** work in practical applications.

Developing interactive games requires proper coordination between user input, graphical rendering, and continuous game updates. Without a structured approach, implementing such real-time applications becomes complex and error-prone.

Therefore, there is a need for a simple and efficient Java-based game system that can:

• Demonstrate real-time graphical interaction
• Handle keyboard-based user input effectively
• Implement collision detection and scoring logic
• Provide a clear understanding of Java Swing and AWT concepts

1.4 Objectives of the Project

To design and develop a **real-time 2D arcade game** using Java:

• To implement **graphical user interface** using Java Swing and AWT
• To control paddle movement using **keyboard events**
• To implement **ball movement and collision detection** with walls, paddle, and bricks
• To generate and manage bricks using a **2D array structure**
• To calculate and display the **score dynamically** during gameplay
• To display **game over and winning messages** based on game conditions
• To apply **object-oriented programming concepts** for structured and modular design

1.5 Scope of the Project

The scope of this project includes the design and development of a **real-time 2D arcade game** using Java. The project focuses on implementing core game functionalities such as **ball movement, paddle control, brick generation, collision detection, scoring, and game state management** using Java Swing and AWT.

This project is intended for **educational and learning purposes**, helping beginners understand graphical programming and event-driven application development in Java. The game runs as a **standalone desktop application** and does not require any external hardware or internet connectivity.

Although the current implementation consists of a single level with basic gameplay mechanics, the project architecture allows future enhancements such as **multiple levels, improved graphics, sound effects, power-ups, and additional game features**.

CHAPTER 2: SYSTEM OVERVIEW

2.1 Overview of the System

The **Breakout Ball Game System** is a Java-based desktop application designed to provide an interactive 2D gaming experience. The system simulates real-time gameplay where a ball moves continuously, interacts with a paddle controlled by the user, and breaks bricks arranged on the screen.

The system is developed using **Java Swing and AWT**, which handle graphical rendering, keyboard input, and real-time updates. The gameplay logic is managed using a **timer-based mechanism**, allowing smooth and continuous movement of game elements.

Unlike traditional console-based Java programs, this system operates in a **graphical and event-driven environment**, enabling real-time user interaction. The modular design of the system ensures clear separation of responsibilities among different classes, improving readability, maintainability, and scalability of the application.

2.2 Functional Description

The overall working of the Breakout Ball Game system can be described through the following functional stages:

1. **Game Initialization:**
   When the application is launched, the game window is created using JFrame, and the gameplay panel is loaded. Initial positions of the ball, paddle, and bricks are set.
2. **User Input Handling:**
   The system continuously listens for keyboard inputs using the KeyListener interface. The left and right arrow keys are used to move the paddle horizontally.
3. **Game Loop Execution:**
   A Timer is used to update the game state at regular intervals. This ensures smooth and continuous movement of the ball during gameplay.

4. **Collision Detection:**

The system checks for collisions between:

- Ball and walls
- Ball and paddle
- Ball and bricks

  When a collision occurs, the direction of the ball is changed accordingly.

5. **Score Management:**

Each time the ball hits and breaks a brick, the brick is removed and the score is increased dynamically.

6. **Game Status Evaluation:**

The system continuously checks the game conditions. If all bricks are destroyed, a **win message** is displayed. If the ball falls below the paddle, a **game over message** is shown.

2.3 System Modules

The Breakout Ball Game system is divided into the following major modules to ensure a structured and modular design.

2.3.1 Game Initialization Module

This module is responsible for initializing the game environment. It creates the main application window using JFrame and sets up the gameplay panel. Initial values such as the position of the ball, paddle, bricks, and score are defined in this module. It ensures that the game starts in a ready state when the application is launched.

2.3.2 User Input Module

This module handles keyboard-based user interactions. It uses the `KeyListener` interface to detect left and right arrow key presses. Based on the user input, the paddle is moved horizontally on the screen. This module enables real-time interaction between the player and the game.

2.3.3 Game Logic Module

The game logic module controls the movement of the ball and manages collision detection with walls, paddle, and bricks. It updates the ball's direction based on collision events and ensures smooth gameplay using a timer-based execution mechanism.

2.3.4 Brick Management Module

This module is responsible for generating and managing the bricks using a 2D array structure. It draws the bricks on the screen and removes them when hit by the ball. This module also helps in tracking the remaining bricks in the game.

2.3.5 Score and Game Status Module

This module manages the scoring system and evaluates the game state. It updates the score whenever a brick is broken and displays appropriate messages such as **Game Over** or **You Win** based on the game conditions.

2.3 Data Flow Description

The data flow within the Breakout Ball Game system begins with game initialization and continues through user interaction and graphical updates. When the application starts, the initial game data such as ball position, paddle position, brick layout, and score are loaded into the system.

User input data in the form of keyboard actions is captured through the `KeyListener` interface and processed by the gameplay logic. Based on these inputs, the paddle position is updated in real time. Simultaneously, the ball's position is continuously updated using a timer-based game loop.

Collision data between the ball, paddle, walls, and bricks is processed to determine changes in ball direction and brick removal. The updated game state, including score and game status, is then rendered graphically on the screen for user visualization.

This continuous data flow ensures smooth gameplay, real-time interaction, and accurate game state updates without any external dependencies.

2.4 Advantages of the Proposed System

Provides a real-time and interactive gaming experience
- Helps beginners understand Java Swing and AWT concepts clearly
- Demonstrates effective use of event handling and timers
- Simple, lightweight, and easy to execute on any system with Java installed
- Does not require internet connectivity or external hardware
- Modular design allows easy modification and future enhancements
- Enhances logical thinking and problem-solving skills through game development

CHAPTER 3: HARDWARE REQUIREMENTS

Since the **Breakout Ball Game** is a **Java-based desktop application**, it does not require any external hardware components. The game runs entirely on a computer system using Java Swing and AWT library

3.1 Hardware Requirements

The minimum hardware requirements for developing and running the project are:

• Computer or Laptop
• Processor: Intel i3 or higher (or equivalent)
• RAM: Minimum 4 GB
• Storage: At least 500 MB free disk space
• Input Devices: Keyboard
• Display: Monitor

3.2 Software Requirements

The software requirements for this project are:

• Operating System: Windows / Linux / macOS
• Programming Language: Java (JDK 8 or above)
• IDE: **Visual Studio Code (VS Code)**
• Java Libraries:
– Java Swing
– Java AWT

3.3 Development Environment (Visual Studio Code)

Visual Studio Code is used as the development environment for this project. It provides a lightweight and efficient platform for writing, debugging, and running Java programs.

Role in the Project:
• Writing and editing Java source code
• Compiling and running the game
• Debugging and fixing errors
• Managing project files and folders

VS Code is chosen due to its **simplicity, speed, and support for Java extensions**.

3.4 Execution Requirements

• Java Development Kit (JDK) must be installed
• Java extensions should be enabled in VS Code
• The project should be compiled and executed using the terminal or Run option

3.5 Advantages of Using VS Code

• Lightweight and fast IDE
• Easy to use for beginners
• Good Java support through extensions
• Cross-platform compatibility

3.6 Limitations of the System

• The game consists of a **single level** with fixed brick layout
• Graphics are **basic** and do not include advanced animations
• No sound effects are included in the current version
• The game does not support **pause and resume** functionality
• Difficulty level remains constant throughout the gameplay

CHAPTER 4: SOFTWARE REQUIREMENTS

This chapter describes the software tools and technologies used for designing, developing, and executing the **Breakout Ball Game**. The selected software components ensure smooth game development, graphical rendering, and real-time user interaction..

4.1 Operating System

The project can be developed and executed on the following operating systems:

• Windows 10 / 11
• Linux (Ubuntu)
• macOS

These operating systems support Java and Visual Studio Code, which are required to develop and run the application.

4.2 Java Programming Language

Java is the primary programming language used to develop the Breakout Ball Game.

Features:
• Object-oriented and platform-independent
• Strong support for GUI development
• Robust and secure language
• Easy to maintain and modify

Role in the Project:
• Implements game logic and rules
• Handles object-oriented structure
• Controls ball, paddle, and brick behavior
• Manages scoring and game states

4.3 Java Development Kit (JDK)

The Java Development Kit (JDK) is required to compile and execute Java programs.

Role in the Project:
• Compiles Java source code
• Executes the game application
• Provides required Java libraries

Version Used: JDK 8 or above

4.4 Visual Studio Code (VS Code)

Visual Studio Code is used as the Integrated Development Environment (IDE) for this project.

Features:
• Lightweight and fast editor
• Supports Java through extensions
• Built-in terminal and debugging support
• Easy file and project management

Role in the Project:
• Writing and editing Java source code
• Compiling and running the game
• Debugging errors
• Managing project structure

4.5 Java Swing and AWT Libraries

Java Swing and AWT are used for graphical user interface development.

Java Swing:
• Used to create windows, panels, and UI components
• Provides event handling support

Java AWT:
• Used for drawing shapes, colors, and graphics
• Handles keyboard events

Role in the Project:
• Rendering game graphics
• Handling keyboard input
• Displaying score and messages

4.6 Supporting Tools

The following supporting tools are used:

• Command Prompt / Terminal – For compiling and running Java files
• Java Extensions in VS Code – For enhanced Java development support

CHAPTER 5: SYSTEM ARCHITECTURE

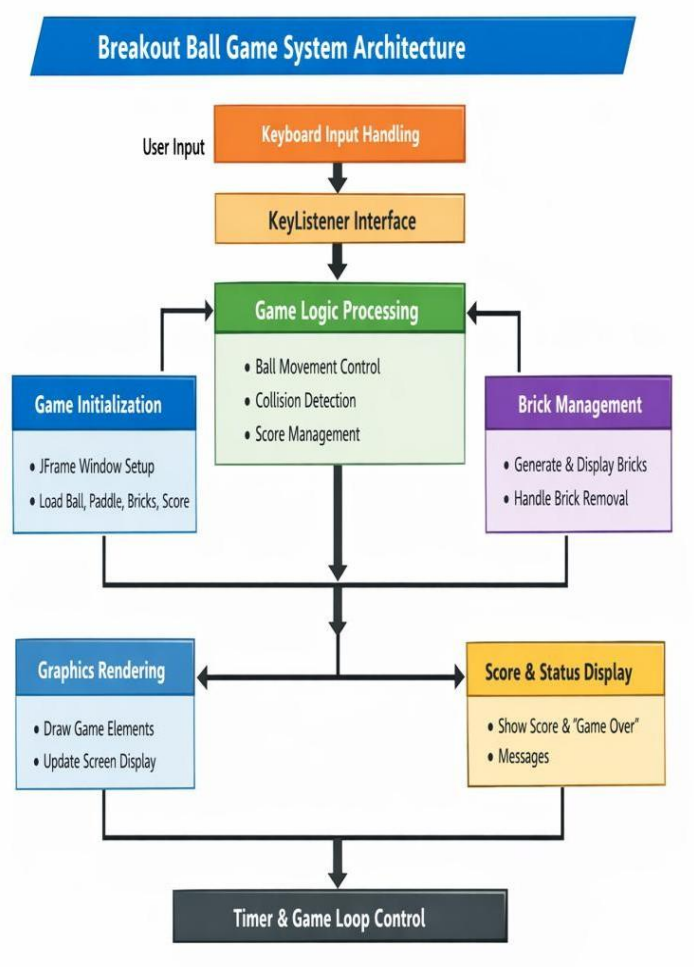
This chapter describes the architectural design of the **Breakout Ball Game** developed using Java. The system architecture explains how different software components interact with each other to provide real-time gameplay, handle user input, and manage game logic and graphics.

The architecture follows a **modular and event-driven design**, where each component is responsible for a specific task such as game initialization, user interaction, game logic processing, and graphical rendering. Java Swing and AWT are used to create the graphical interface and handle events, while a timer-based mechanism ensures continuous game updates.

5.1 Architectural Overview

The proposed system follows a **modular and event-driven architecture model** that enables efficient game initialization, user interaction, game logic processing, and graphical rendering. The architecture is designed to handle all game computations locally within the Java application, eliminating the need for any external services or internet connectivity.

Each module operates independently while interacting through well-defined interfaces, ensuring smooth real-time gameplay and easy maintainability. This architectural design allows the game to respond instantly to user inputs and supports future enhancements such as additional levels and improved graphics.



System Architecture Layers

The system architecture consists of four major layers:

1. Input Layer
2. Game Logic Layer
3. Rendering Layer
4. Application Control Layer

Each layer performs a specific function and interacts seamlessly with the others.

5.2 Input Layer

The Input Layer is responsible for capturing user interactions during gameplay. It detects keyboard inputs such as left and right arrow key movements to control the paddle.

This layer continuously listens for user actions and forwards the input commands to the game logic for processing.

5.3 Game Logic Layer

The Game Logic Layer is the core of the Breakout Ball Game. It handles all game rules and mechanics, including:

- Ball movement and direction
- Paddle movement
- Brick collision detection
- Score calculation
- Life count management
- Game over and win conditions

This layer updates the game state in real time based on user input and collision events.

5.4 Rendering Layer

The Rendering Layer is responsible for displaying all visual elements of the game. It renders:

- Ball
- Paddle
- Bricks
- Score and lives
- Game over and win messages

Java's Graphics and Swing libraries are used to draw and refresh the game components on the screen at regular intervals.

5.5 Application Control Layer

The Application Control Layer manages the overall execution of the game. It controls:

- Game initialization
- Game loop execution

- Level restarting
- Pause and resume functionality
- Application termination

This layer ensures smooth coordination between input handling, game logic processing, and rendering.

5.6 Data Flow Description

The data flow in the Breakout Ball Game follows a sequential and interactive process:

1. User input is captured through keyboard actions (left and right arrow keys).
2. The input commands are sent to the game controller module.
3. The game logic processes the input and updates paddle movement.
4. Ball position and direction are recalculated based on collisions with walls, paddle, and bricks.
5. Collision detection logic updates brick status, score, and remaining lives.
6. The updated game state is passed to the rendering module.
7. The screen is refreshed to display the updated positions of the ball, paddle, bricks, score, and game messages.

This continuous data flow ensures smooth gameplay, real-time responsiveness, and accurate game state updates throughout execution.

5.7 Advantages of the Proposed Architecture

• Simple and modular architecture makes the game easy to understand and maintain
• Real-time processing ensures smooth and responsive gameplay
• Clear separation of game logic, input handling, and rendering improves code readability
• Efficient collision detection provides accurate gameplay behavior
• Scalable design allows easy addition of new levels, features, and enhancements
• Platform-independent Java implementation enables execution on multiple operating systems

CHAPTER 6: PROJECT ALGORITHM

This chapter explains the step-by-step procedure followed in the **Breakout Ball Game** developed using Java. The game involves controlling a paddle to bounce a ball and break bricks while tracking score and game status..

6.1 Algorithm Description

The project algorithm defines the logical flow of operations involved in running the **Breakout Ball Game**. The algorithm is designed to execute continuously in real time, handling user input, updating game objects, detecting collisions, and rendering graphics smoothly using Java Swing.

Initially, the game environment is set up by creating the main window, initializing the paddle, ball, brick layout, and score variables. A timer is used to control continuous execution of the game loop.

During gameplay, the algorithm constantly listens for keyboard inputs to move the paddle left or right. Simultaneously, the ball's position is updated based on its current direction and speed. Collision detection logic checks interactions between the ball and walls, paddle, and bricks.

When the ball collides with a brick, the brick is removed, the ball direction is reversed, and the score is incremented. Boundary conditions are monitored to determine game over or winning scenarios. The game screen is refreshed repeatedly to reflect updated positions and scores.

This algorithm ensures smooth gameplay, real-time interaction, and accurate game logic execution until the player either clears all bricks or loses the ball

Algorithm: Breakout Ball Game

1. Start the application
2. Create the main game window using `JFrame`
3. Initialize the `Gameplay` panel
4. Initialize paddle position
5. Initialize ball position and movement direction
6. Generate brick layout using `MapGenerator` class
7. Initialize score and game status variables
8. Start the game timer
9. Listen for keyboard input
10. If LEFT arrow key is pressed, move paddle left
11. If RIGHT arrow key is pressed, move paddle right
12. Update ball position continuously
13. Check collision of ball with left and right walls
14. Reverse ball's horizontal direction if wall collision occurs
15. Check collision of ball with top wall
16. Reverse ball's vertical direction if collision occurs
17. Check collision of ball with paddle
18. Check collision of ball with bricks

If collision occurs:

19. Remove the brick
20. Reverse ball direction
21. Increase score
22. Check if all bricks are destroyed
23. If yes, display **"You Won"** message and stop the game
24. Check if ball crosses bottom boundary
25. If yes, display **"Game Over"** message and stop the game
26. Repaint the game screen continuously
27. Repeat steps until the game ends
28. End

6.2 Pseudo Code

Gameplay.java

```
import java.util.*;
import java.awt.event.*;

import javax.swing.*;

import java.awt.*;

import javax.swing.*;
import javax.swing.Timer;

public class Gameplay extends JPanel implements KeyListener, ActionListener
{
    private boolean play = true;
    private int score = 0;

    private int totalBricks = 58;

    private Timer timer;
    private int delay=3;

    private int playerX = 310;

    private  int  ballposX  =  250;
    private  int  ballposY  =  350;
    private  int  ballXdir  =  -2;
    private int ballYdir = -3;

    private MapGenerator map;

    public Gameplay()
    {
        map   =   new   MapGenerator(4,   16);
        addKeyListener(this);
        setFocusable(true);
        setFocusTraversalKeysEnabled(false);
        timer=new            Timer(delay,this);
        timer.start();
    }

    public void paint(Graphics g)
    {
        // background
        g.setColor(Color.black);
        g.fillRect(1, 1, 692, 592);

        // drawing map
        map.draw((Graphics2D) g);
```

```
        // borders
        g.setColor(Color.yellow);
        g.fillRect(0, 0, 3, 592);
        g.fillRect(0, 0, 692, 3);
        g.fillRect(691, 0, 3, 592);

        // the scores
        g.setColor(Color.white);
        g.setFont(new Font("serif",Font.BOLD, 25));
        g.drawString(""+score, 590,30);

        // the paddle
        g.setColor(Color.green);
        g.fillRect(playerX, 550, 100, 8);

        // the ball
        g.setColor(Color.yellow);
        g.fillOval(ballposX, ballposY, 20, 20);

        // when you won the game
        if(totalBricks <= 0)
        {
            play = false;
            ballXdir = 0;
            ballYdir = 0;
            g.setColor(Color.RED);
            g.setFont(new Font("serif",Font.BOLD, 30));
            g.drawString("You Won", 260,300);

            g.setColor(Color.RED);
            g.setFont(new      Font("serif",Font.BOLD,      20));
            g.drawString("Press (Enter) to Restart", 230,350);
        }

        // when you lose the game
        if(ballposY > 570)
        {
            play = false;
            ballXdir = 0;
            ballYdir = 0;
            g.setColor(Color.RED);
            g.setFont(new      Font("serif",Font.BOLD,      30));
            g.drawString("Game Over, Scores: "+score, 190,300);

            g.setColor(Color.RED);
            g.setFont(new      Font("serif",Font.BOLD,      20));
            g.drawString("Press (Enter) to Restart", 230,350);
        }

        g.dispose();
    }

    public void keyPressed(KeyEvent e)
```

```
    {
        if (e.getKeyCode() == KeyEvent.VK_RIGHT)
        {
            if(playerX >= 600)
            {
                playerX = 600;
            }
            else
            {
                moveRight();
            }
        }

        if (e.getKeyCode() == KeyEvent.VK_LEFT)
        {
            if(playerX < 10)
            {
                playerX = 10;
            }
            else
            {
                moveLeft();
            }
        }
        if (e.getKeyCode() == KeyEvent.VK_ENTER)
        {
            if(!play)
            {
                play = true;
                ballposX = 120;
                ballposY = 350;
                ballXdir = -1;
                ballYdir = -2;
                playerX = 310;
                score = 0;
                totalBricks = 21;
                map = new MapGenerator(3, 7);

                repaint();
            }
        }
    }

    public void keyReleased(KeyEvent e) {}
    public void keyTyped(KeyEvent e) {}

    public void moveRight()
    {
        play = true;
        playerX+=20;
    }

    public void moveLeft()
```

```
    {
        play = true;
        playerX-=20;
    }

    public void actionPerformed(ActionEvent e)
    {
        timer.start();
        if(play)
        {
            if(new     Rectangle(ballposX,     ballposY,     20,     20).intersects(new
Rectangle(playerX, 550, 30, 8)))
            {
                ballYdir = -ballYdir;
                ballXdir = -2;
            }
            else    if(new    Rectangle(ballposX,    ballposY,    20,    20).intersects(new
Rectangle(playerX + 70, 550, 30, 8)))
            {
                ballYdir = -ballYdir;
                ballXdir = ballXdir + 1;
            }
            else    if(new    Rectangle(ballposX,    ballposY,    20,    20).intersects(new
Rectangle(playerX + 30, 550, 40, 8)))
            {
                ballYdir = -ballYdir;
            }

            // check map collision with the ball
            A: for(int i = 0; i<map.map.length; i++)
            {
                for(int j =0; j<map.map[0].length; j++)
                {
                    if(map.map[i][j] > 0)
                    {
                        //scores++;
                        int brickX = j * map.brickWidth + 80;
                        int brickY = i * map.brickHeight + 50;
                        int brickWidth = map.brickWidth;
                        int brickHeight = map.brickHeight;

                        Rectangle rect = new Rectangle(brickX, brickY, brickWidth,
brickHeight);
                        Rectangle ballRect = new Rectangle(ballposX, ballposY, 20, 20);
                        Rectangle brickRect = rect;

                        if(ballRect.intersects(brickRect))
                        {
                            map.setBrickValue(0, i, j);
                            score+=5;
                            totalBricks--;

                            // when ball hit right or left of brick
```

```
                            if(ballposX + 19 <= brickRect.x || ballposX + 1 >=
brickRect.x + brickRect.width)
                            {
                                ballXdir = -ballXdir;
                            }
                            // when ball hits top or bottom of brick
                            else
                            {
                                ballYdir = -ballYdir;
                            }

                            break A;
                        }
                    }
                }
            }

            ballposX += ballXdir;
            ballposY += ballYdir;

            if(ballposX < 0)
            {
                ballXdir = -ballXdir;
            }
            if(ballposY < 0)
            {
                ballYdir = -ballYdir;
            }
            if(ballposX > 670)
            {
                ballXdir = -ballXdir;
            }

            repaint();
        }
    }
}
```

Main.java

```
import javax.swing.*;

public class Main {
    public static void main(String[] args) {
        JFrame obj = new JFrame();
        Gameplay gamePlay = new Gameplay();

        obj.setBounds(10,10,700,600);
        obj.setTitle("Breakout Ball Game");
        obj.setResizable(false);
        obj.setVisible(true);

        obj.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        obj.add(gamePlay);
        obj.setVisible(true);
    }
}
```

6.3 Key Characteristics of the Algorithm

• **Real-time execution**
The algorithm runs continuously using a timer to update ball movement, paddle position, and game state in real time.

• **Event-driven processing**
Keyboard events are used to control paddle movement, ensuring quick and responsive user interaction.

• **Efficient collision detection**
Lightweight collision logic is used to detect interactions between the ball, walls, paddle, and bricks.

• **Smooth graphics rendering**
Java Swing repaint mechanism ensures smooth and flicker-free screen updates.

• **Score tracking and game state control**

The algorithm dynamically updates score, win condition, and game over status.

• **Simple and optimized logic**
Uses basic mathematical calculations, making it efficient and suitable for beginner-level Java game development.

• **Continuous gameplay loop**
The game loop executes until all bricks are destroyed or the player loses the ball.

CHAPTER 7: IMPLEMENTATION

This chapter describes the implementation of the **Breakout Ball Game** using Java Swing and AWT. It explains how the game window is created, how game components such as the paddle, ball, and bricks are implemented, and how the game logic is executed in real time.

The implementation focuses on handling keyboard inputs, ball movement, collision detection, score calculation, and game state control using object-oriented programming principles. Java's `Timer` class is used to manage continuous gameplay, while graphical rendering is handled using the `Graphics` class.

The overall implementation ensures smooth gameplay, responsive controls, and accurate game behavior from start to end.

Gameplay.Class

```
mport java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Rectangle;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
import javax.swing.JPanel;
import javax.swing.Timer;

public class Gameplay extends JPanel implements KeyListener, ActionListener {
   private boolean play = true;
   private int score = 0;
   private int totalBricks = 58;
   private Timer timer;
   private int delay = 3;
   private int playerX = 310;
   private int ballposX = 250;
   private int ballposY = 350;
   private int ballXdir = -2;
   private int ballYdir = -3;
   private MapGenerator map = new MapGenerator(4, 16);

   public Gameplay() {
      this.addKeyListener(this);
      this.setFocusable(true);
      this.setFocusTraversalKeysEnabled(false);
      this.timer = new Timer(this.delay, this);
      this.timer.start();
   }
```

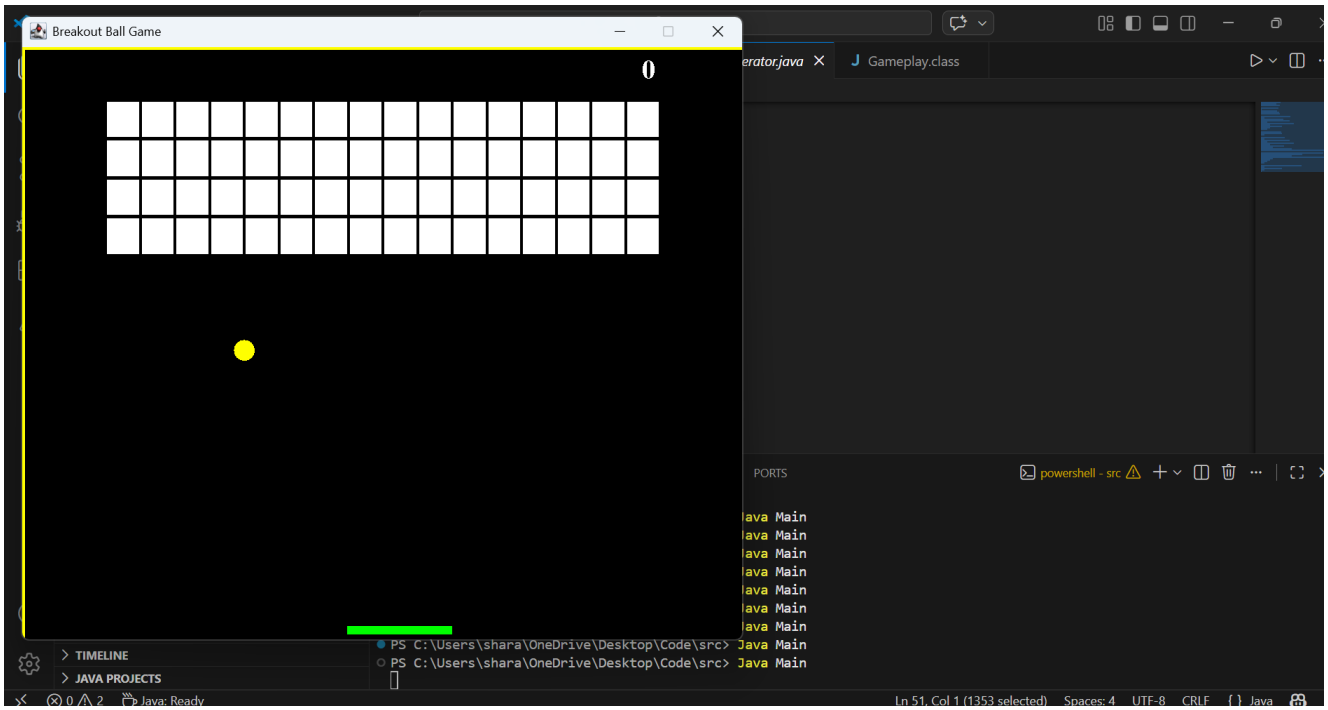
```
public void paint(Graphics var1) {
    var1.setColor(Color.black);
    var1.fillRect(1, 1, 692, 592);
    this.map.draw((Graphics2D)var1);
    var1.setColor(Color.yellow);
    var1.fillRect(0, 0, 3, 592);
    var1.fillRect(0, 0, 692, 3);
    var1.fillRect(691, 0, 3, 592);
    var1.setColor(Color.white);
    var1.setFont(new Font("serif", 1, 25));
    var1.drawString("" + this.score, 590, 30);
    var1.setColor(Color.green);
    var1.fillRect(this.playerX, 550, 100, 8);
    var1.setColor(Color.yellow);
    var1.fillOval(this.ballposX, this.ballposY, 20, 20);
    if (this.totalBricks <= 0) {
        this.play = false;
        this.ballXdir = 0;
        this.ballYdir = 0;
        var1.setColor(Color.RED);
        var1.setFont(new Font("serif", 1, 30));
        var1.drawString("You Won", 260, 300);
        var1.setColor(Color.RED);
        var1.setFont(new Font("serif", 1, 20));
        var1.drawString("Press (Enter) to Restart", 230, 350);
    }

    if (this.ballposY > 570) {
        this.play = false;
        this.ballXdir = 0;
        this.ballYdir = 0;
        var1.setColor(Color.RED);
        var1.setFont(new Font("serif", 1, 30));
        var1.drawString("Game Over, Scores: " + this.score, 190, 300);
        var1.setColor(Color.RED);
        var1.setFont(new Font("serif", 1, 20));
        var1.drawString("Press (Enter) to Restart", 230, 350);
    }

    var1.dispose();
}

public void keyPressed(KeyEvent var1) {
    if (var1.getKeyCode() == 39) {
        if (this.playerX >= 600) {
            this.playerX = 600;
        } else {
            this.moveRight();
        }
    }

    if (var1.getKeyCode() == 37) {
        if (this.playerX < 10) {
```

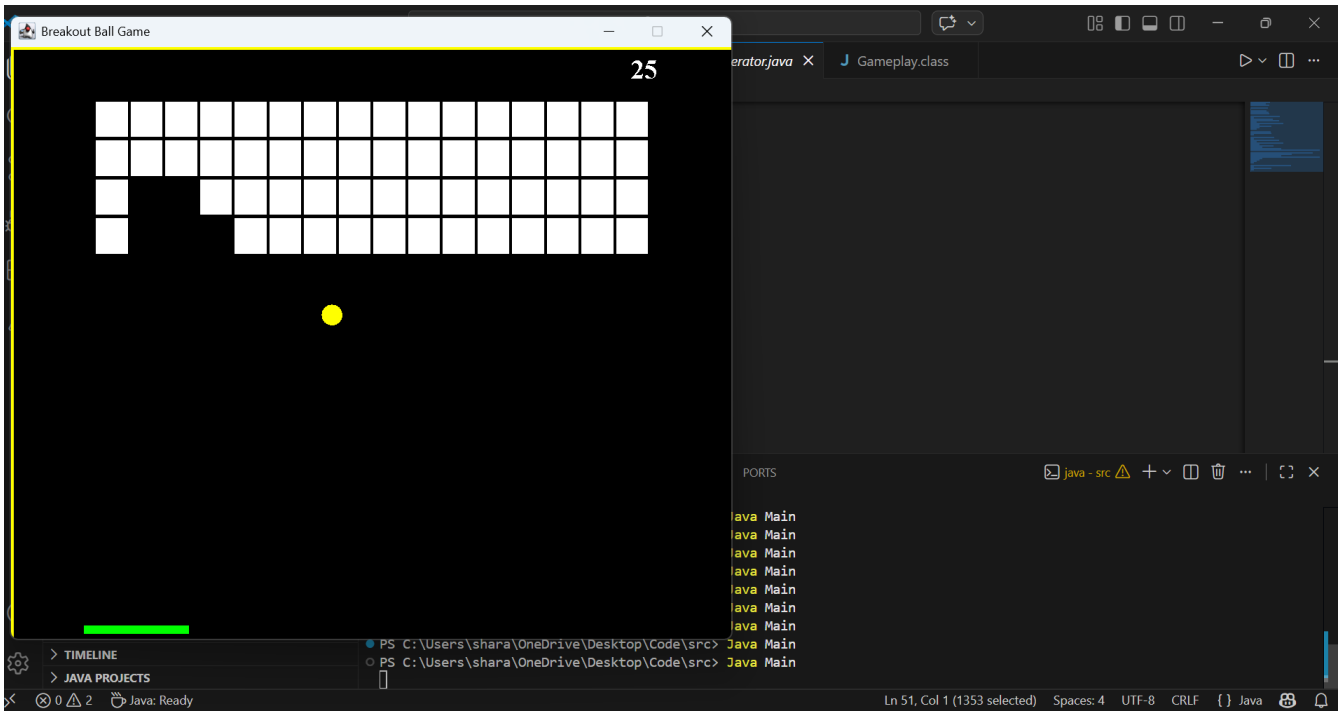
```
            this.playerX = 10;
         } else {
            this.moveLeft();
         }
      }

      if (var1.getKeyCode() == 10 && !this.play) {
         this.play = true;
         this.ballposX = 120;
         this.ballposY = 350;
         this.ballXdir = -1;
         this.ballYdir = -2;
         this.playerX = 310;
         this.score = 0;
         this.totalBricks = 21;
         this.map = new MapGenerator(3, 7);
         this.repaint();
      }

   }

   public void keyReleased(KeyEvent var1) {
   }

   public void keyTyped(KeyEvent var1) {
   }

   public void moveRight() {
      this.play = true;
      this.playerX += 20;
   }

   public void moveLeft() {
      this.play = true;
      this.playerX -= 20;
   }

   public void actionPerformed(ActionEvent var1) {
      this.timer.start();
      if (this.play) {
         if   ((new   Rectangle(this.ballposX,   this.ballposY,   20,   20)).intersects(new
Rectangle(this.playerX, 550, 30, 8))) {
            this.ballYdir = -this.ballYdir;
            this.ballXdir = -2;
         } else if ((new Rectangle(this.ballposX, this.ballposY, 20, 20)).intersects(new
Rectangle(this.playerX + 70, 550, 30, 8))) {
            this.ballYdir = -this.ballYdir;
            ++this.ballXdir;
         } else if ((new Rectangle(this.ballposX, this.ballposY, 20, 20)).intersects(new
Rectangle(this.playerX + 30, 550, 40, 8))) {
            this.ballYdir = -this.ballYdir;
         }
```

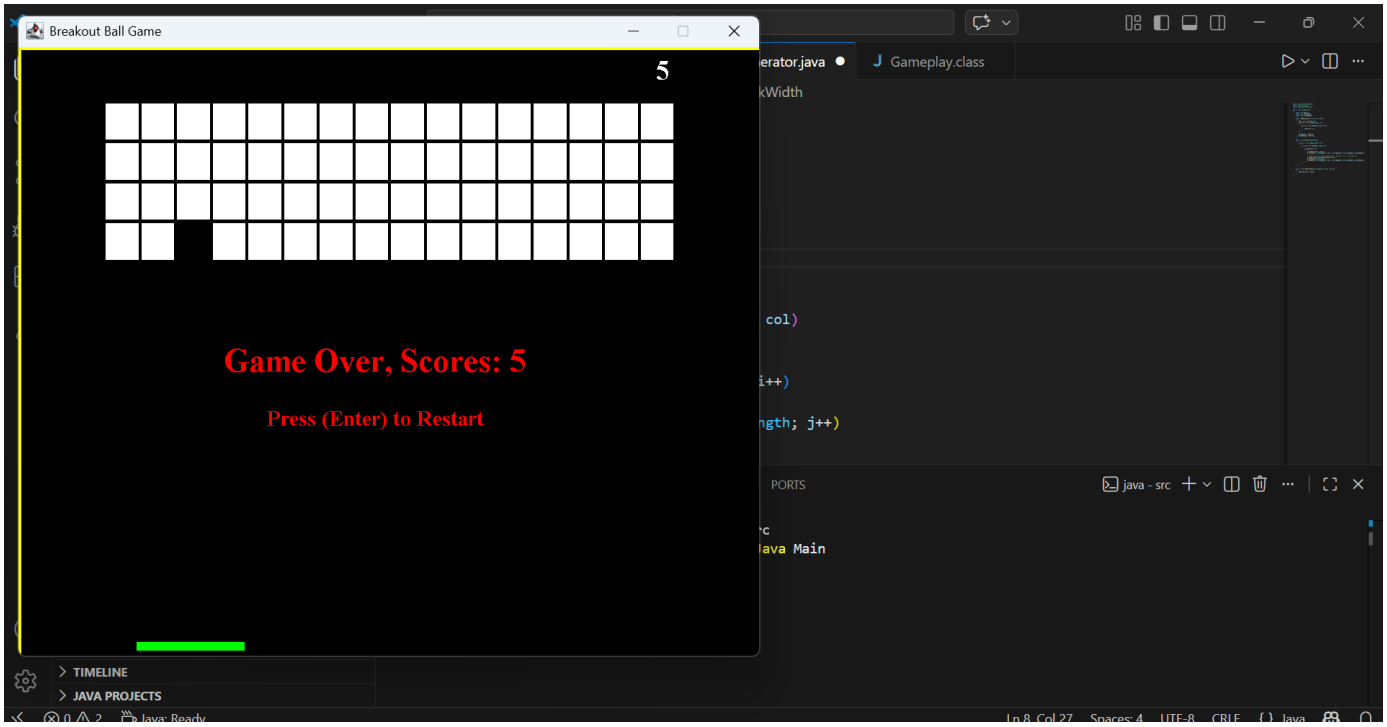
```
        label56:
        for(int var2 = 0; var2 < this.map.map.length; ++var2) {
            for(int var3 = 0; var3 < this.map.map[0].length; ++var3) {
                if (this.map.map[var2][var3] > 0) {
                    int var4 = var3 * this.map.brickWidth + 80;
                    int var5 = var2 * this.map.brickHeight + 50;
                    int var6 = this.map.brickWidth;
                    int var7 = this.map.brickHeight;
                    Rectangle var8 = new Rectangle(var4, var5, var6, var7);
                    Rectangle var9 = new Rectangle(this.ballposX, this.ballposY, 20, 20);
                    if (var9.intersects(var8)) {
                        this.map.setBrickValue(0, var2, var3);
                        this.score += 5;
                        --this.totalBricks;
                        if (this.ballposX + 19 > var8.x && this.ballposX + 1 < var8.x +
var8.width) {
                            this.ballYdir = -this.ballYdir;
                            break label56;
                        }

                        this.ballXdir = -this.ballXdir;
                        break label56;
                    }
                }
            }
        }

        this.ballposX += this.ballXdir;
        this.ballposY += this.ballYdir;
        if (this.ballposX < 0) {
            this.ballXdir = -this.ballXdir;
        }

        if (this.ballposY < 0) {
            this.ballYdir = -this.ballYdir;
        }

        if (this.ballposX > 670) {
            this.ballXdir = -this.ballXdir;
        }

        this.repaint();
    }

    }
}
```

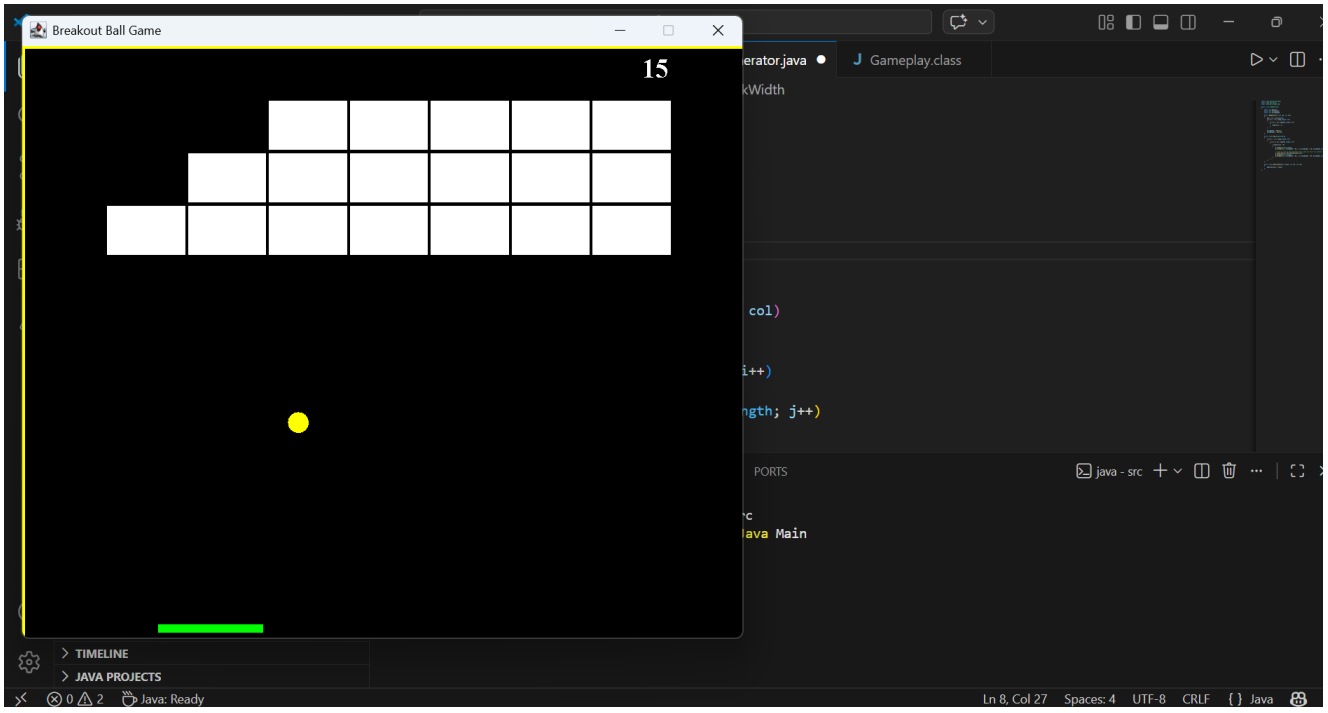
MapGenerator.java

```
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Graphics2D;

public class MapGenerator
{
    public int map[][];
    public int brickWidth;
    public int brickHeight;

    public MapGenerator (int row, int col)
    {
        map = new int[row][col];
        for(int i = 0; i<map.length; i++)
        {
            for(int j =0; j<map[0].length; j++)
            {
                map[i][j] = 1;
            }
        }

        brickWidth = 540/col;
        brickHeight = 150/row;
    }

    public void draw(Graphics2D g)
    {
        for(int i = 0; i<map.length; i++)
        {
            for(int j =0; j<map[0].length; j++)
            {
                if(map[i][j] > 0)
                {
                    g.setColor(Color.white);
                    g.fillRect(j * brickWidth + 80, i * brickHeight + 50, brickWidth,
brickHeight);

                    // this is just to show separate brick, game can still run without it
                    g.setStroke(new BasicStroke(3));
                    g.setColor(Color.black);
                    g.drawRect(j * brickWidth + 80, i * brickHeight + 50, brickWidth,
brickHeight);
                }
            }
        }
    }

    public void setBrickValue(int value, int row, int col)
    {
        map[row][col] = value;
    }
```


Breakout Ball Game
0
Gameplay.class
PORTS
powershell - src
PS C:\Users\shara\OneDrive\Desktop\Code\src> Java Main
PS C:\Users\shara\OneDrive\Desktop\Code\src> Java Main
TIMELINE
JAVA PROJECTS
Java: Ready
Ln 51, Col 1 (1353 selected)
Spaces: 4
UTF-8
CRLF
Java


Breakout Ball Game
25
Gameplay.class
PORTS
java - src
PS C:\Users\shara\OneDrive\Desktop\Code\src> Java Main
PS C:\Users\shara\OneDrive\Desktop\Code\src> Java Main
TIMELINE
JAVA PROJECTS
Java: Ready
Ln 51, Col 1 (1353 selected)
Spaces: 4
UTF-8
CRLF
Java


Breakout Ball Game
5
Game Over, Scores: 5
Press (Enter) to Restart
Gameplay.class
TIMELINE
JAVA PROJECTS


Breakout Ball Game
15
Gameplay.class
TIMELINE
JAVA PROJECTS
Java: Ready
Ln 8, Col 27
Spaces: 4
UTF-8
CRLF
Java

CHAPTER 8: CONCLUSION

The **Breakout Ball Game** successfully demonstrates the use of Java for developing an interactive 2D arcade-style game using **Swing and AWT**. The project effectively applies core Java concepts such as object-oriented programming, event handling, graphics rendering, and timer-based execution to create a smooth and engaging gameplay experience.

The game allows the player to control a paddle using keyboard inputs and break bricks using a bouncing ball. Real-time collision detection between the ball, paddle, walls, and bricks ensures accurate game behavior. The scoring system, win condition, and game over logic enhance the overall interactivity and completeness of the application.

By dividing the program into multiple classes such as Main, Gameplay, and MapGenerator, the project follows a modular and well-structured design approach. This improves code readability, reusability, and maintainability. The use of Java's Timer and Graphics classes ensures continuous game updates and smooth visual output.

Overall, the Breakout Ball Game serves as an excellent mini project for understanding basic game development concepts in Java. It strengthens problem-solving skills and provides hands-on experience with real-time application development. The project can be further enhanced by adding multiple levels, sound effects, improved graphics, and additional gameplay features.